

Basic used in Theory of Computation:

Everything in mathematics is based on symbols. This is also true for automata theory of computation. The symbols are generally letters & digits. The central concept of automata theory include the "alphabet" (a set of symbol), "string" (a list of symbols from an alphabet), and "language" (a set of strings from the same alphabet).

● Alphabet: An alphabet is a finite, non-empty set of symbols or characters. We use the symbol Σ for an alphabet. $\text{ex} \rightarrow$ If Σ is an alphabet containing all the 26 characters used in English lang. then Σ is finite & non-empty set and is represented as $\Sigma = \{a, b, c, \dots, z\}$. Another example is a binary alphabet $\Sigma = \{0, 1\}$. $\Sigma = \{1, 2, 3, \dots\}$ This is not alphabet bcz this is infinite. $B = \{ \}$ This is not alphabet bcz this is empty.

● String or word: A string is a finite sequence of symbols chosen from some fixed alphabet. $\text{ex} \rightarrow$ "abc" is a string over an alphabet $\Sigma = \{a, b, c\}$. Another example is 01101 is a string from the binary alphabet $\Sigma = \{0, 1\}$.

▲ The empty string or null string is denoted by ϵ . The empty string is the string with zero occurrences of symbols.

● Length of a string: The length of a string is the no. of symbols or characters in that string. If w is a string then its length is denoted by $|w|$. $\text{eg: (1) If } w = abcd, \text{ then length of } w \text{ is } |w| = 4.$

(2) If $w = 123$, then length of w is $|w| = 3$.

(3) ϵ is the empty string and has length 0.

①

● Concatenation of strings:

If w_1 and w_2 are two strings then concatenation of w with w_1 is a string and it is denoted by $w_1 w_2$. In other words, we say that w_1 is followed by w_2 and length is $|w_1 w_2| = |w_1| + |w_2|$. for example -
If $w_1 = abc$ and $w_2 = xyz$ then $w_1 w_2 = abcxyz$
 $|w_1| = 3$
 $|w_2| = 3$
 $|w_1 w_2| = 3 + 3 = 6$.

● Reverse of string:

R.O.S can be achieved by flipping over last symbols.
If $w = abc$ then $w^R = cba$, w^R is a reverse of w
 $|w| = |w^R|$.

● Language: A language 'L' denotes a set of strings over set certain alphabet. The lang ϕ is undefined as similar to infinity which is undefined & this is similar to an empty box $\{ \}$ i.e., a lang. without an string. for example:

- 1) $L_1 = \{ ab, aabb, aaabbb \}$ is a lang over alphabet $\Sigma = \{a, b\}$
- 2) $L_2 = \{ \epsilon, a, aa, aaa, aaaa, \dots \}$ is a lang over alphabet $\Sigma = \{a, b\}$
- 3) $L_3 = \{ a^n b^n c^n \mid n \geq 1 \}$ is a lang.
- 4) $L_4 = \{ a^n b^n c^m \mid m, n \geq 1 \}$ is a lang.
- 5) lang. of all string consists of 'n' 0's followed by 'n' 1's for some $n \geq 0$.
 $L = \{ \epsilon, 01, 0011, 000111, \dots \}$
- 6) The set of strings over 0's and 1's with equal no. of each.
 $L = \{ \epsilon, 01, 1001, 0101, 011010, \dots \}$

Note: $\phi \neq \{ \epsilon \}$.

The former has ~~one~~ no string & later has one string.

Concatenation of language:

Concatenation of lang. can be defined as :- If L_1 & L_2 are two lang then their concatenation is $L = L_1 L_2$ where $L = \{w \mid w = xy, x \in L_1, y \in L_2\}$ i.e., the string in $L_1 L_2$ formed by choosing a string L_1 and followed by a string in L_2 in all possible combination.

Kleene closure (or Reflexive Transitive closure):

Let Σ be some alphabet then Kleene closure of Σ is denoted by Σ^* also known as RT closure. The length of Kleene closure of any alphabet is infinite. It is defined as follows -

$$\begin{aligned}\Sigma^* &= \{\text{set of all words over } \Sigma\} \\ &= \{\text{word of length zero, words of length one, words of length two, } \dots\} = \bigcup_k \Sigma^k \\ &= \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \dots\end{aligned}$$

for example -

$$1) \text{ If } \Sigma = \{0, 1\}, \text{ then } \Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \Sigma^3 \dots$$

$$\Sigma^0 = \{\epsilon\}$$

$$\Sigma^1 = \{0, 1\}$$

$$\Sigma^2 = \{00, 01, 10, 11\} \text{ and so on.}$$

$$\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, \dots\}$$

$$2) \text{ If } \Sigma = \{a\}, \text{ then}$$

$$\Sigma^* = \{\epsilon, a, aa, aaa, aaaa, \dots\}$$

Positive closure: If Σ is an alphabet then positive closure of Σ is denoted by Σ^+ & defined as follows

$$\Sigma^+ = \Sigma^* - \{\epsilon\}$$

$$= \{\text{set of all words over } \Sigma \text{ excluding empty string } \epsilon\}$$

(13)

for example -

(1) = If $\Sigma = \{a\}$ then $\Sigma^+ = \{a, aa, aaa, aaaa, \dots\}$.

(2) = If $\Sigma = \{0, 1\}$ then $\Sigma^+ = \{0, 1, 00, 11, 01, 10, \dots\}$.

● Formal Languages :

To define the concept of a formal lang., we need the idea of an alphabet and string. We have seen the Kleene closure & positive closure. A formal lang. L on the alphabet Σ is a subset of Σ^+ i.e., $L \subseteq \Sigma^+$. Here we are defining a formal lang. L as a collection of strings (or words).

● GRAMMAR :-

Grammar is nothing but set of rules used to define the lang. These rules can be written with the help of terminal or non-terminal.

Example - $S \rightarrow 1S | 0S$
 $S \rightarrow 0 | 1$

Here terminal symbols are 0 and 1. The non-terminal symbol is S .

$S \rightarrow 1S$
 $\rightarrow 10S$
 $\rightarrow 101S$
 $\rightarrow 1010$
 $\rightarrow 1011$

$L = \{0, 1, 00, 11, 10, 01, 1010, 1011, \dots\}$

● What is Automata ?

The term "Automata" is derived from the Greek Word "AUTOMATA" which means "self-acting".

An automaton (Automata in plural) is an abstract self-propelled computing device which follows a pre-defined sequence of operations automatically.

An automata is finite no. of states is c/d finite Automata (FA) or finite state m/c (FSM).

● Description of Finite Automata:

A finite automata consist of a finite memory c/d input take tape, a read only head and finite control. The I/P is provided on the tape and head reads one symbol on the tape and move forward. The movement is decided by the finite control when input is exhausted the finite decides the Validity (for acceptability) of the accept input by acceptance or rejection (yes/no). It does not write anything which is a limitation of this model. The FA ~~model~~ ^{model of FA} is shown below.

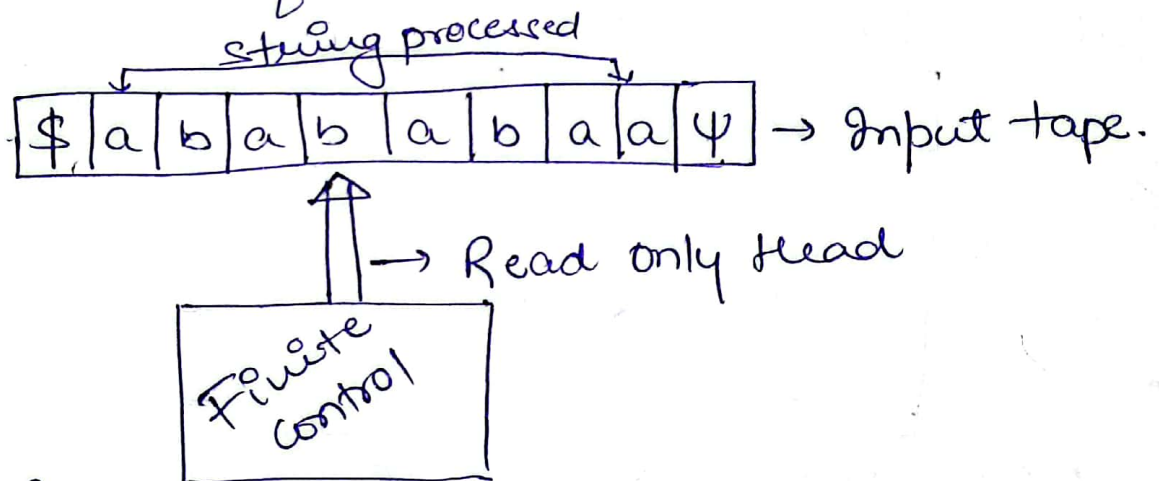


fig: Model of FA.

The input tape is divided into cells & each cell contains one symbol from the input alphabet Σ . The symbol dollar is used at the left most cell & the symbol ψ is used at the right most cell. to indicate the beginning and end of the input tape. The head reads one symbol on the input tape and finite control controls the next configuration (state). The head cannot

write and cannot move backward once the input symbol ^{is has been} read. So finite Automata cannot remember its previous reads symbols. This is one of the major limitation of FA, which shows that it cannot be used in such computation where knowledge of previous input is necessary. FA is also known as FSM or FSA.

• Types of FA Automata.

There are two types of FA.

- 1) Deterministic FA
- 2) Non-Deterministic FA.

Ques Draw the Transition diagram for Transition Table shown below.

Soln

δ / Σ	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_2	q_0
$*q_2$	q_2	q_2

Fig: Transition Table

A DFA; $M = (Q, \Sigma, \delta, q_0, F)$

$Q = \{q_0, q_1, q_2\}$

$F = \{q_2\}$

$\Sigma = \{a, b\}$

$q_0 = \{q_0\}$

$\delta(q_0, a) = q_1$

$\delta(q_0, b) = q_2$

$\delta(q_1, a) = q_2$

$\delta(q_1, b) = q_0$

$\delta(q_2, a) = q_2$

$\delta(q_2, b) = q_2$

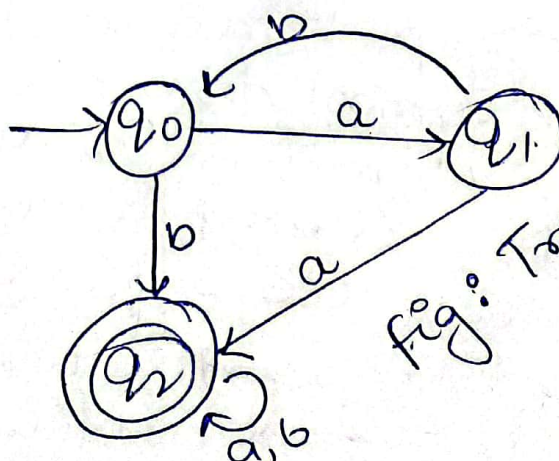
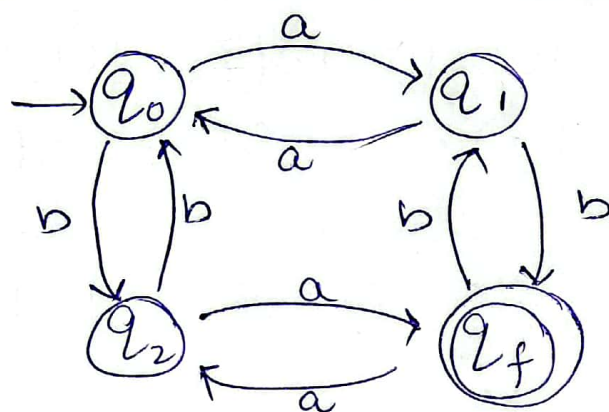


Fig: Transition Diagram

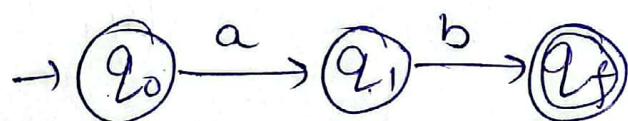
(6)

Acceptability of a string by DFA:

Let a DFA $M = (Q, \Sigma, \delta, q_0, F)$ and input string $w \in \Sigma^*$. The string w is accepted by DFA M if and only if $\delta(q_0, w) = q_f$ where $q_f \in F$. In case w is accepted by M , the execution of string w ends in a final state. Consider the DFA shown in figure and input strings ① ab ② abb check the acceptability of each string.

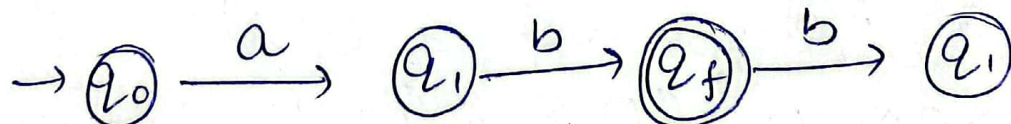


(1) Solution: let string $w_1 = ab$, then the transition sequence is as follows:



Execution ends in final state q_f , hence string "ab" is accepted.

(2) Solution: let string $w_2 = abb$, then the transition sequence is as follows:



Execution ends not in final state q_f , hence string "abb" is not accepted.

Consider the finite automata, $M = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, q_0)$ and the transition table is -

(11)

δ/Σ	0	1
$\rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

1) find the acceptability of the following string

- (a) 110101
(b) 101100

(a) let $w_1 = 110101$

$$\begin{aligned} \delta(q_0, w_1) &\Rightarrow \delta(q_0, 110101) \Rightarrow \delta(q_1, 10101) \Rightarrow \delta(q_0, 0101) \\ &\Rightarrow \delta(q_2, 101) \Rightarrow \delta(q_3, 01) \Rightarrow \delta(q_1, 1) \Rightarrow q_0 \end{aligned}$$

It means that the string 110101 accepted.

(b) let $w_2 = 101100$

$$\begin{aligned} \delta(q_0, w_2) &\Rightarrow \delta(q_0, 101100) \Rightarrow \delta(q_1, 01100) \Rightarrow \delta(q_3, 1100) \\ &\Rightarrow \delta(q_2, 100) \Rightarrow \delta(q_3, 00) \Rightarrow \delta(q_1, 0) \Rightarrow q_3. \end{aligned}$$

This is not accepted 101100 string.

The language of DFA:

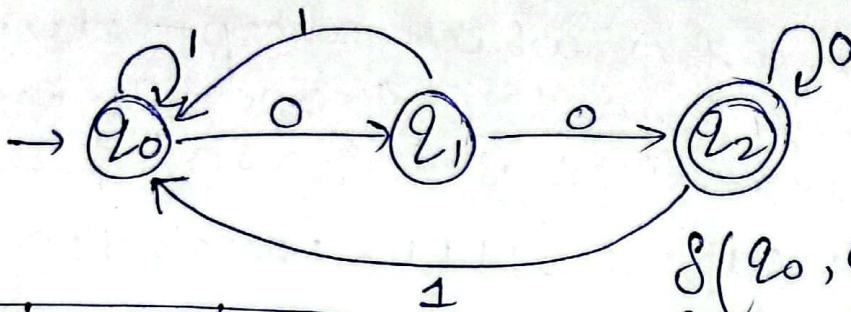
Ques Design a finite Automata that accept set of string such that every string end in 00 over alphabet $\Sigma = \{0, 1\}$

Sol: let DFA, $M = (Q, \Sigma, \delta, q_0, F)$
 $q_0 = \{q_0\}$ (initial state)
 $\Sigma = \{0, 1\}$ is given.

$$L = \{00, 100, 0100, 0110100, 110100, \dots\}$$

$$Q = \{q_0, q_1, q_2\}$$

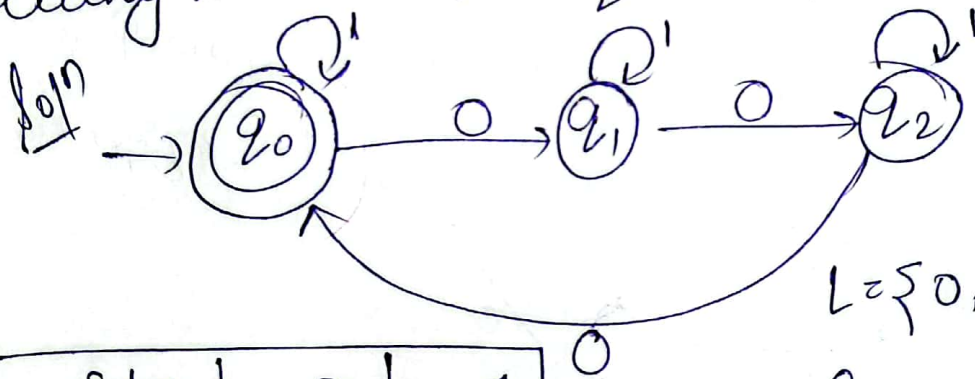
$$F = \{q_0\}$$



δ / Σ	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_0
$*q_2$	q_2	q_0

$$\begin{aligned}\delta(q_0, 0) &= q_1 \\ \delta(q_0, 1) &= q_0 \\ \delta(q_1, 0) &= q_2 \\ \delta(q_1, 1) &= q_0 \\ \delta(q_2, 0) &= q_2 \\ \delta(q_2, 1) &= q_0\end{aligned}$$

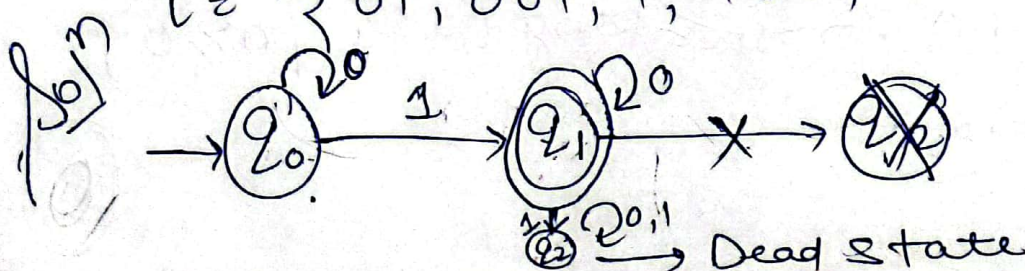
Ques Draw a finite Automata that accepts set of string where the no. of zero in every string is multiple of 3 over alphabet $\Sigma = \{0, 1\}$



δ / Σ	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_2	q_0
$*q_2$	q_0	q_2

$$\begin{aligned}L &= \{0, 000, 1000, 0100, \dots\} \\ Q_0 &= \{q_0\} \\ Q &= \{q_0, q_1, q_2\} \\ F &= \{q_0\}\end{aligned}$$

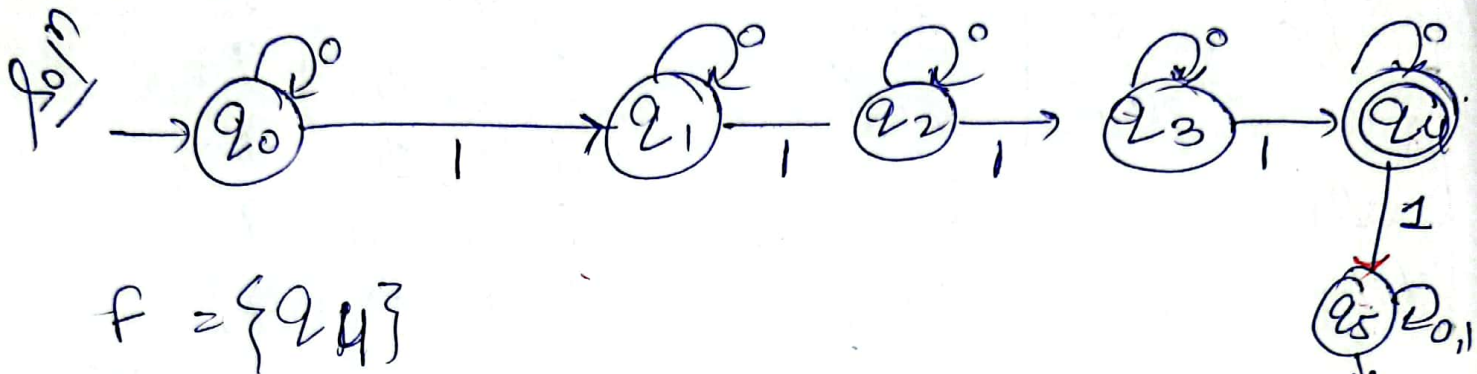
Ques Design a F.A that accepts strings containing exactly one one over alphabet $\Sigma = \{0, 1\}$.



(a)

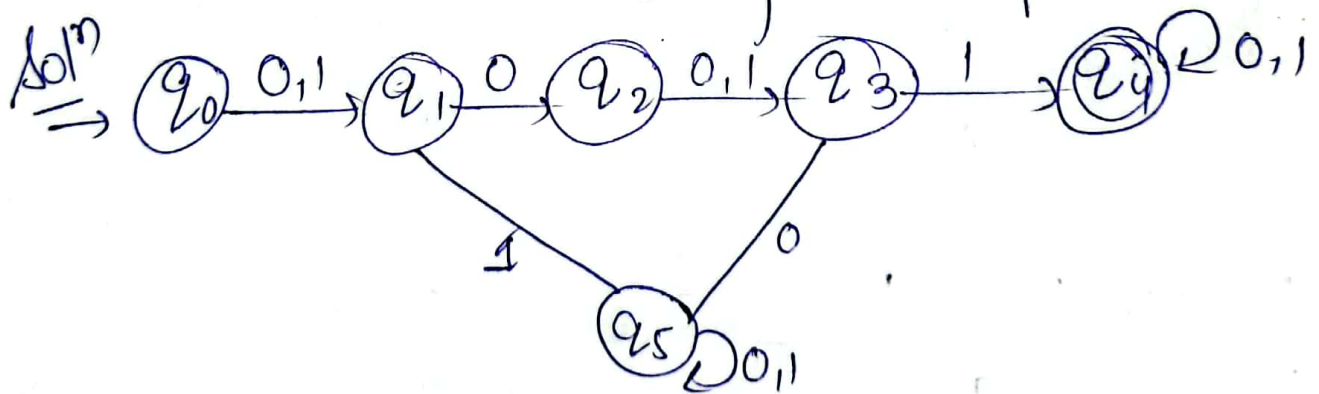
Ques Design a f.A which accept strings containing exactly 4 ones in every string over alphabet $\Sigma = \{0,1\}$.

$L = \{1111, 11011, 001111, 10101010, \dots\}$



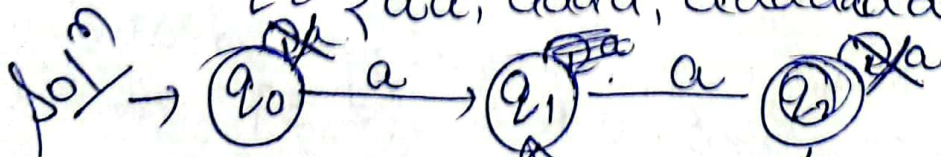
Ques Design a FA which accept the lang.

$L = \{w \in \{0,1\}^* \mid \text{second symbol of } w \text{ is } 0 \text{ and fourth symbol is } 1\}$.

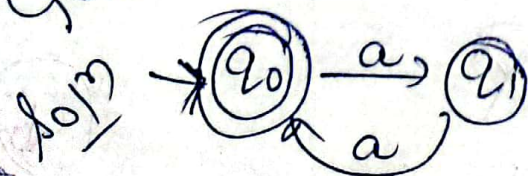


Ques Draw a FA for the lang $L = \{a^{2^n} \mid n \geq 1\}$.

$L = \{aa, aaaa, aaaaaaaa, \dots\}$



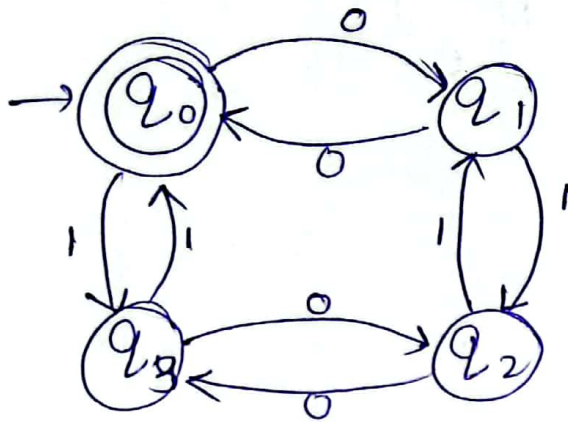
Ques Draw a FA for the lang $L = \{a^{2^n} \mid n \geq 0\}$



Ques Construct a DFA for the following over alphabet $\Sigma = \{0, 1\}$.

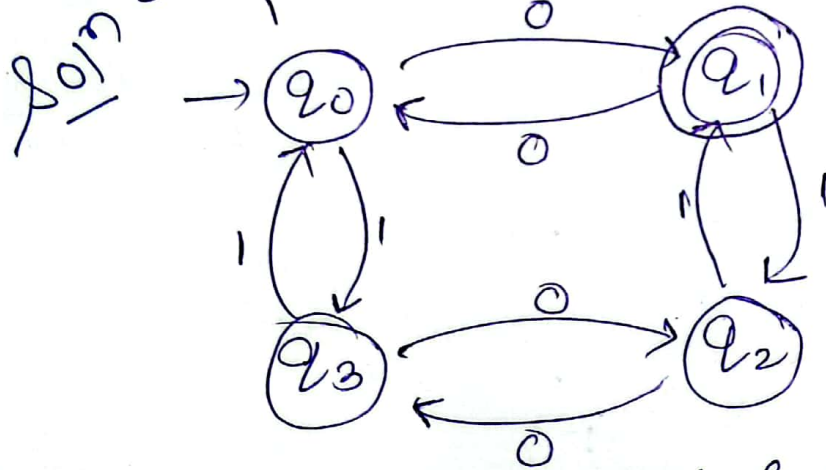
① Even no. of 1's and even no. of 0's

Soln $L = \{0101, 1010, \dots\}$

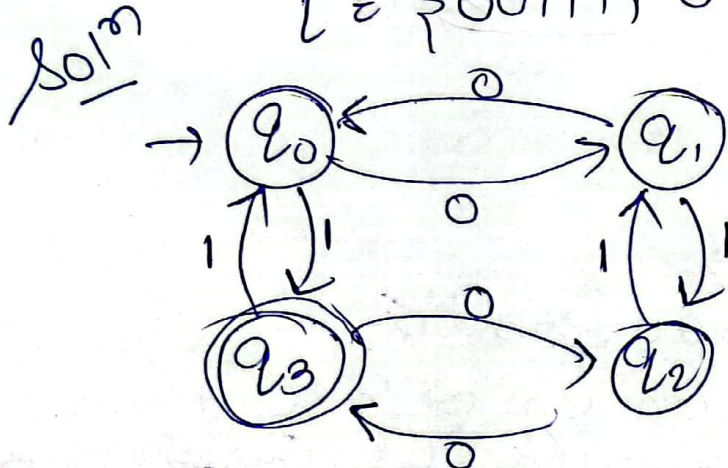


Accepted.

② Even no. of 1's and odd no. of 0's.
 $L = \{11000, 10100, 110, 101, \dots\}$

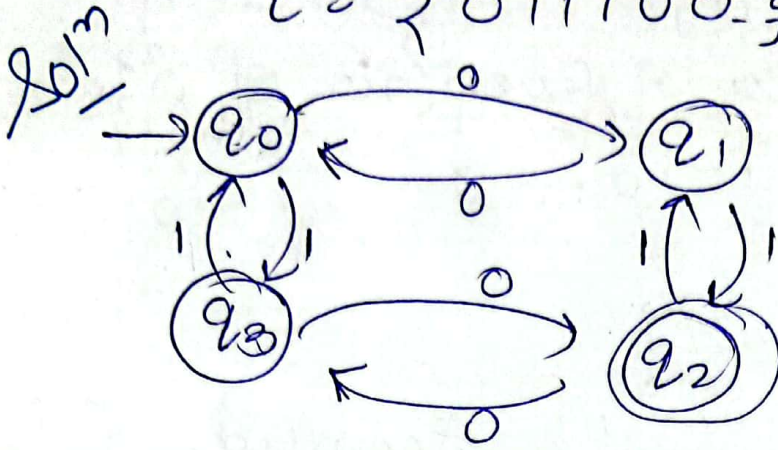


③ Odd no. of 1's and even no. of 0's
 $L = \{00111, 0101100, \dots\}$

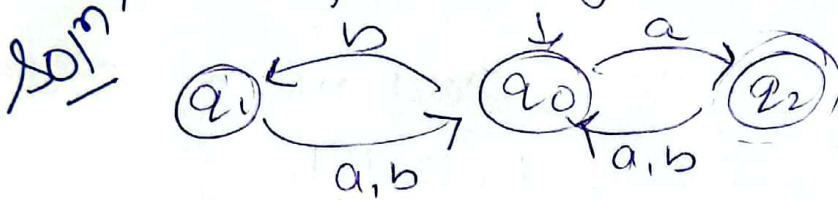


④ Odd no. of 1's and odd no. of 0's. (12)

$$L = \{011100, \dots\}$$

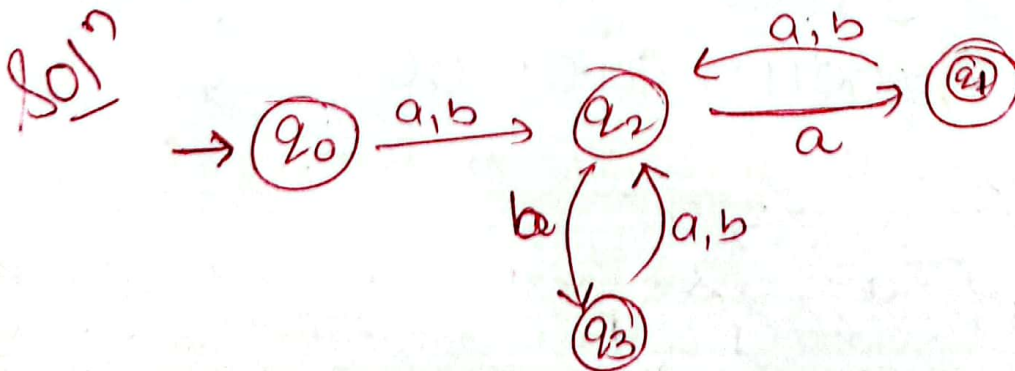


Ques Describe in english the lang. accepted by the following DFA. $L = \{a, aao, baabbaa\}$



The lang. consisting of all the strings that are of odd length and ending with a over alphabet $\Sigma = \{a, b\}$

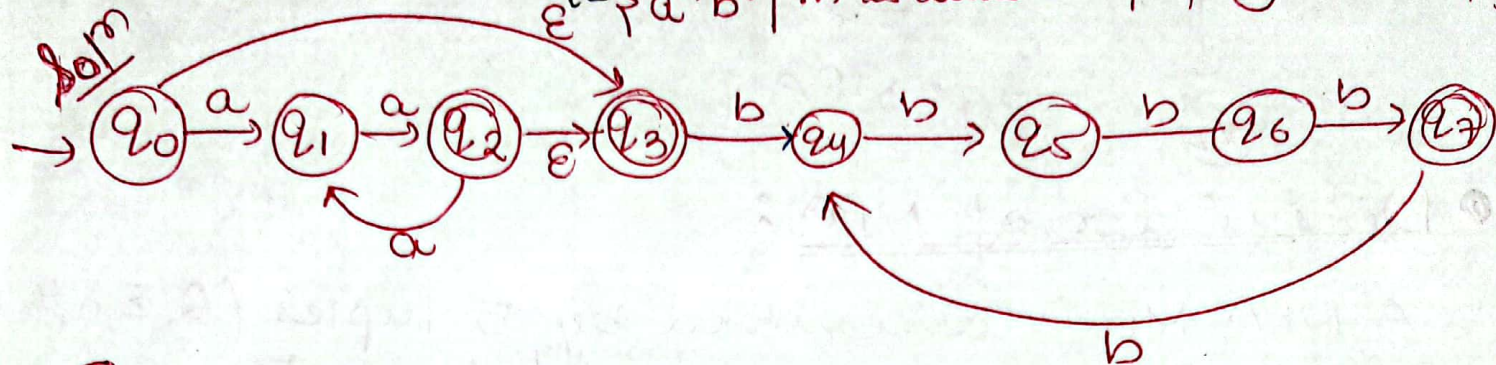
Ques Describe in english lang accepted by the following DFA.



$$L = \{aa, ba, abaa, bbaa, \dots\}$$

The language consisting of all the strings that are of even length and consisting ending with a over alphabet $\Sigma = \{a, b\}$.

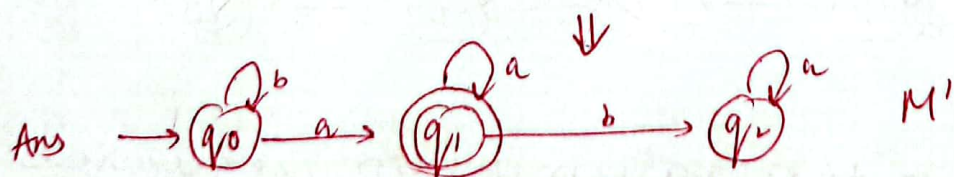
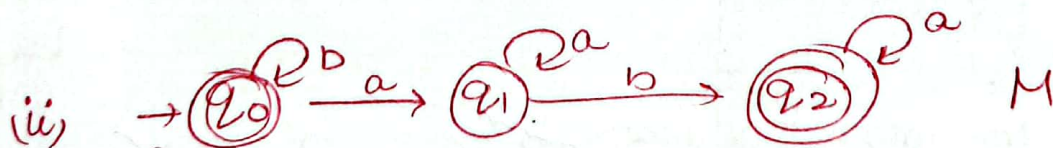
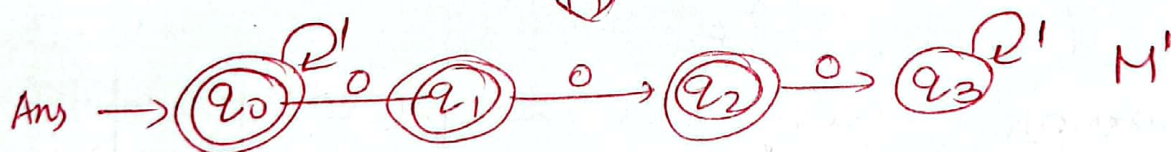
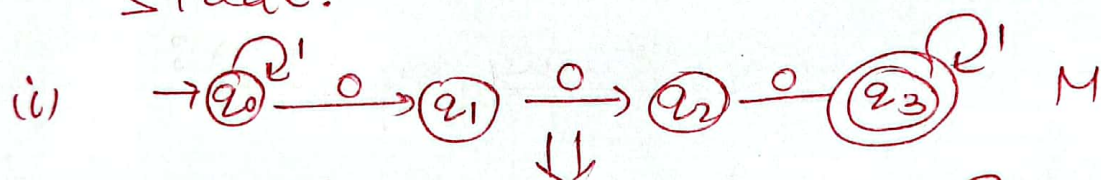
Ques Construct a DFA for the following lang.
 $E = \{a^m b^n \mid m \text{ is divisible by 2} \text{ \& } n \text{ is divisible by 4}\}$



Ques Given a DFA, suggest a procedure to draw DFA which ^{accept} the complement of the lang acceptable by M .

Procedure: ① change the final state to non-final state.

② change the non-final state to final state.



● Non-deterministic Finite Automata:

The concept of NFA is exactly reverse of DFA. The finite Automata is called NFA when $\forall$ many paths for a specific input from current state to next state.

(13)

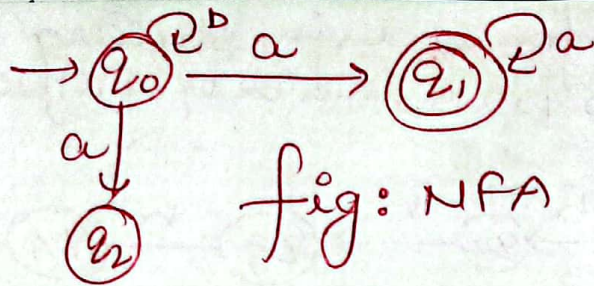


fig: NFA

● Definition of NFA :-

A NFA M_1 is described by 5 tuples $(Q, \Sigma, \delta, q_0, F)$ where Q is finite and ^{non empty} set of state. It contains all the states, Σ is finite and non empty ^{set} of input alphabets, δ is transition funⁿ which maps $Q \times \Sigma \rightarrow 2^Q$, q_0 belongs to Q is the initial state or starting state and F is subset of Q ($F \subseteq Q$) is a set of final state.

● Acceptability of a string by NFA :-

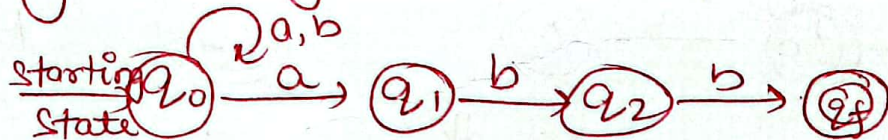
$$\delta(q_0, w) = q_f$$

for example: check the acceptability of following string -

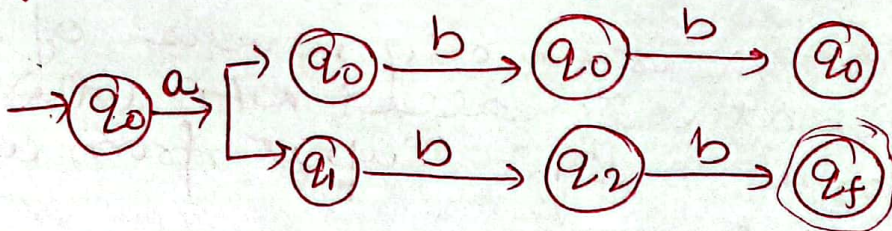
(a) abb

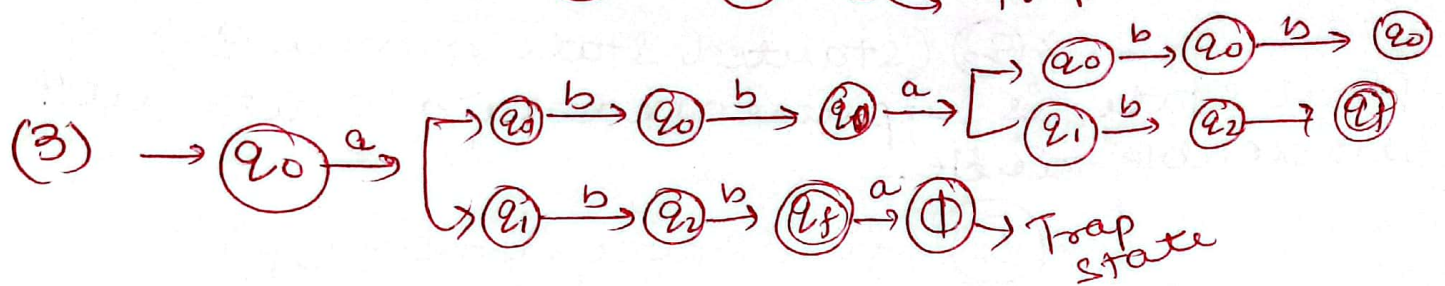
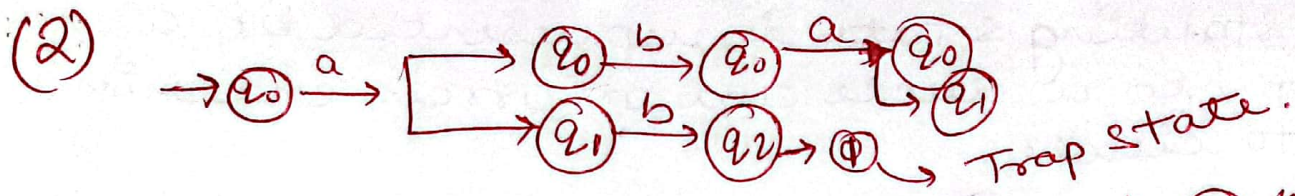
(b) aba

(c) abbabb



1) let $w_1 = abb$, then the transmission sequence for the input string "abb" is as follows:





● Deterministic finite Automata (DFA):

The finite automata is called deterministic finite automata if there is only one path for a specific input from current state to next state.

Example: The DFA can be shown as below:

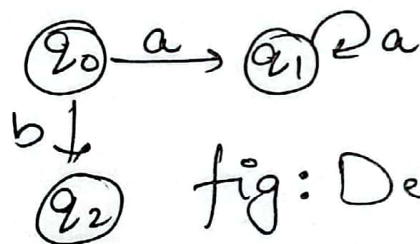


fig: Deterministic finite Automata

● Definition of DFA:

A DFA M is described by five tuples $(Q, \Sigma, \delta, q_0, f)$. Q is finite, non-empty set of states. It contains all the states. Σ is an input alphabet. δ is transition func which map $Q \times \Sigma \rightarrow Q$. q_0 belongs to Q , $q_0 \in Q$ is the initial or starting state. F known as set of final state ($F \subseteq Q$). Simplified notations used in finite automata.

● State Transition Graph:

We represent a finite automata by describing all five tuples $(Q, \Sigma, \delta, q_0, f)$. Transition graph(system) is used for representing finite automata. We used following notations for representing the finite automata.

* The starting state is represented by a state with ϵ in a circle and an arrow entering into circle.

→ (q_0) (started state named q_0)

* Final state is represented by a state with ϵ in double circle.

(q_f) (final state named q_f)

* The Rang or trap state is represented by the symbol Φ within a circle. (Φ)

* Other states are represented by the state name within a circle.

(q_i)

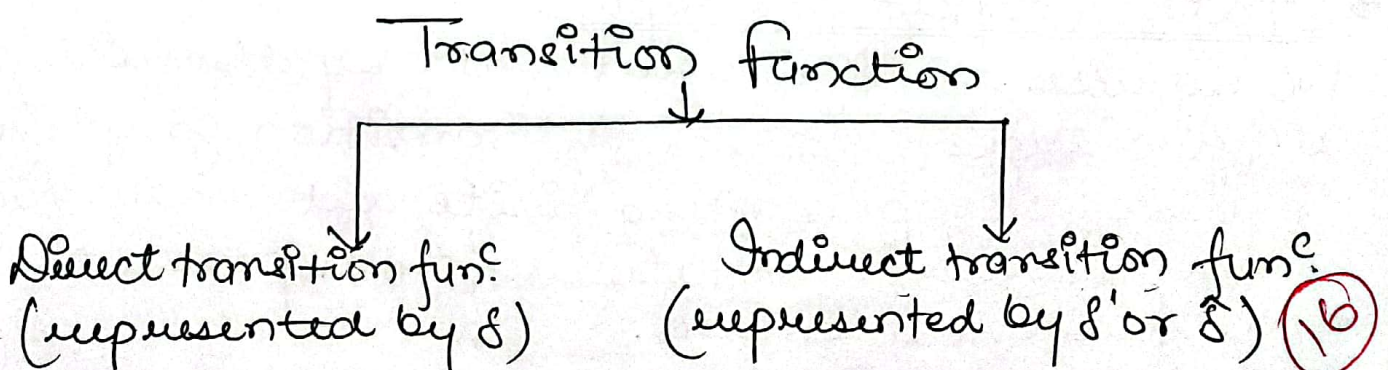
* A directed edge or arc with label shows the transition (or move)

▲ Suppose p is the present state and q is next state on input symbol ' a ' then this is represented by

$$(p) \xrightarrow{a} (q) \text{ or } [\delta(p, a) = q]$$

Transition function:

We have, two types of transition funcⁿ based on the input. If length of the input is zero or one then direct transition otherwise indirect transition.



● Direct Transition function :

When the input is single symbol, transition function is known as direct transition function. It is also known as one step transition ($\delta(p, a) = q$)

● Indirect Transition function :

When the input is a string, transition function is known as indirect transition function. It is also known as one step or more than one step transition function. $\delta(p, w) = q$, where p is the present state & q is the next state after (w) transitions.

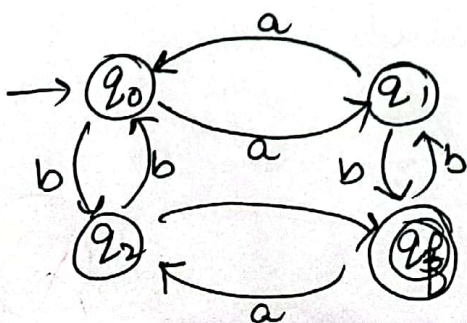
Example : $w = aab$

$$\begin{aligned}\delta(q_0, aab) &= \delta(\delta(q_0, a), ab) \\ &= \delta(q_1, ab) \\ &= \delta(\delta(q_1, a), b) \\ &= \delta(q_2, b) \\ &= q_3.\end{aligned}$$

● Transition Table :

A transition table is a tabular representation of a transition function that takes two arguments and returns a state. The rows of the table corresponds to the states and the columns corresponds to the inputs. The entry for one row corresponding to state q and the column corresponding to input 'a' is the state $\delta(q, a)$.

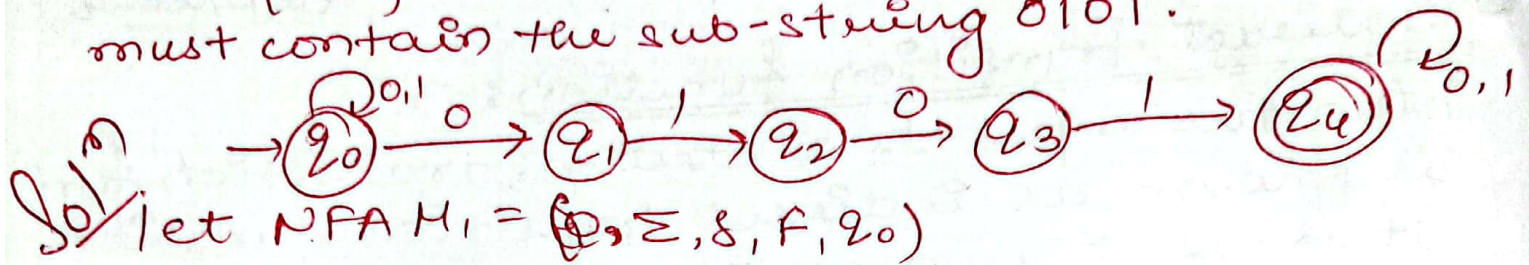
Example : A DFA $M = (\{q_0, q_1, q_2, q_3\}, \{a, b\}, \delta, q_0, \{q_3\})$ is shown in figure. The graph shown in fig is known as transition graph.



δ / Σ	a	b
$\rightarrow q_0$	q_1	q_2
q_1	q_0	q_3
q_2	q_3	q_0
$*q_3$	q_2	q_1

Language of NFA:

Ques Design a NFA for the lang. L over alphabet $\Sigma = \{0, 1\}$ such that every string in the L must contain the sub-string 0101.



$Q = \{q_0, q_1, q_2, q_3, q_4\}$

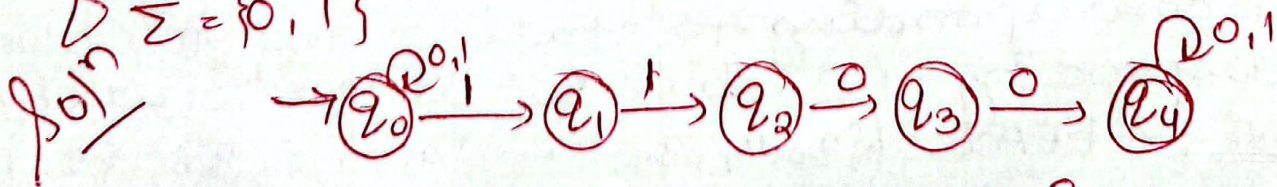
$q_0 = \{q_0\}$ initial state

$F = \{q_4\}$ Final state

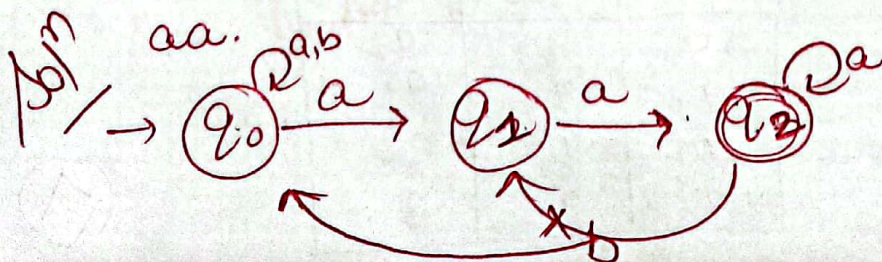
$\Sigma = \{0, 1\}$

δ/Σ	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	—	$\{q_2\}$
q_2	q_3	—
q_3	—	q_4
$\neq q_4$	q_4	q_4

Ques Construct a NFA in which double one is followed by double zero over alphabet $\Sigma = \{0, 1\}$



Ques Design a NFA to accept strings with a's and b's such that the string end with aa.

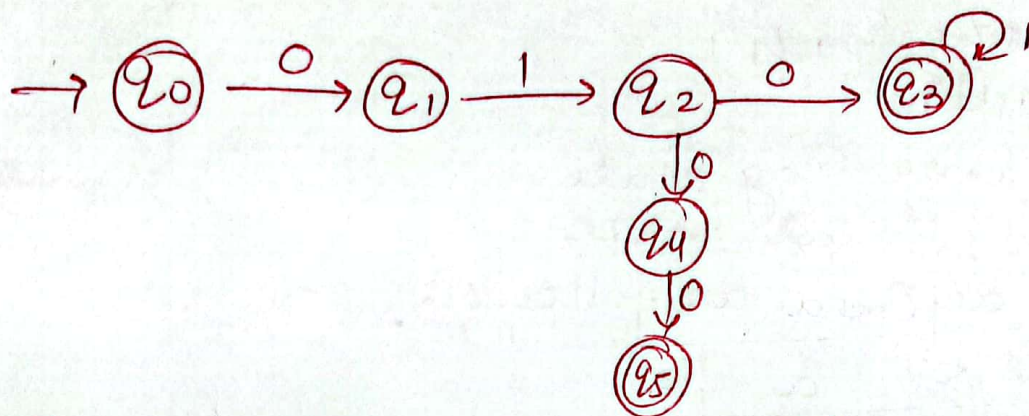


Ques construct NFA for the lang.

$$L = \{0101^n \cup 0100 \mid n \geq 0\}$$

$$L = \{010, 0101, 01011, 010111, \dots, 0100\}$$

solⁿ



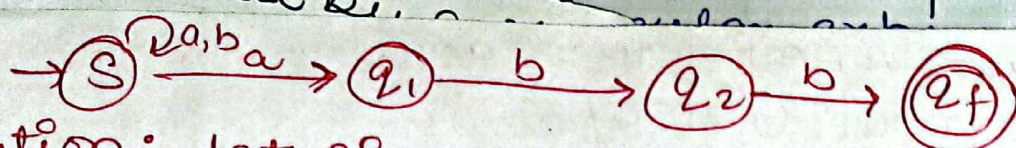
Difference b/w DFA and NFA

S.No	NFA	DFA	INDEX
01.	Same	Same	Power
02.	$Q \times \Sigma \rightarrow 2^Q$ (states 0, 1 or more)	$Q \times \Sigma \rightarrow Q$ (only 1 or 0)	Transition function
03.	The time needed for executing an input string is generally more as compared to DFA. Due to Non-Deterministic	The time needed for executing an input string is less as compared to NFA	Time Complexity
04.	All DFA are NFA but not all NFA are DFA.	All are DFA are NFA	Supremacy

● Construction of equivalent DFA 'M' from given NFA M1.

Ques Consider a NFA shown in fig.

19



Solution: let given NFA, $M = (Q, \Sigma, \delta, S, F)$
 and its equivalent DFA, $M_1 = (Q_1, \Sigma, \delta_1, [S], F_1)$
 $Q = \{S, q_1, q_2, q_f\}$

$\Sigma = \{a, b\}$

$S = \{S\}$, starting state

$F = \{q_f\}$, final state

δ is defined as follows:

δ/Σ	a	b
$\rightarrow S$	$\{S, q_1\}$	$\{S\}$
q_1	—	$\{q_2\}$
q_2	—	$\{q_f\}$
$*q_f$	—	—

δ_1 is defined as follows:

δ_1/Σ	a	b
$\rightarrow [S]$	$[S, q_1]$	$[S]$
$[S, q_1]$	$[S, q_1]$	$[S, q_2]$
$[S, q_2]$	$[S, q_1]$	$[S, q_f]$
$*[S, q_f]$	$[S, q_1]$	$[S]$

$$* \delta_1[[S, q_1], a] =$$

$$[\delta(S, a) \cup \delta(q_1, a)]$$

$$= [S, q_1] \cup \emptyset$$

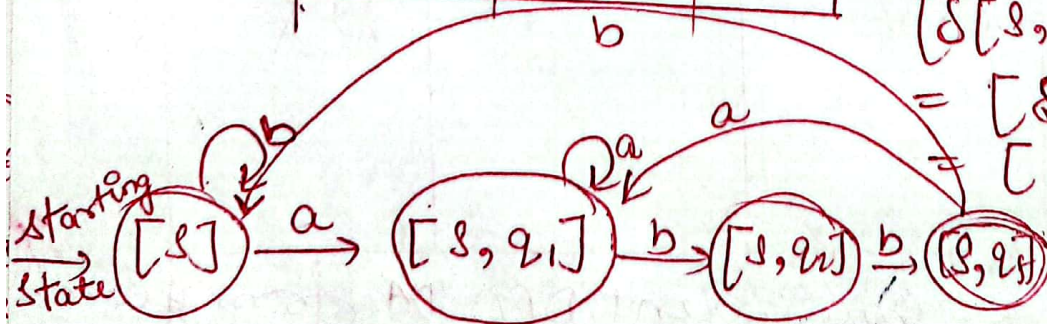
$$= [S, q_1]$$

$$* \delta_1[[S, q_1], b] =$$

$$[\delta(S, b) \cup \delta(q_1, b)]$$

$$= [S] \cup \{q_2\}$$

$$= [S, q_2]$$



$S = \{[S]\}$ starting state

$\Sigma = \{a, b\}$

$F_1 = \{[S, q_f]\}$

$Q_1 = \{[S], [S, q_1], [S, q_2], [S, q_f]\}$

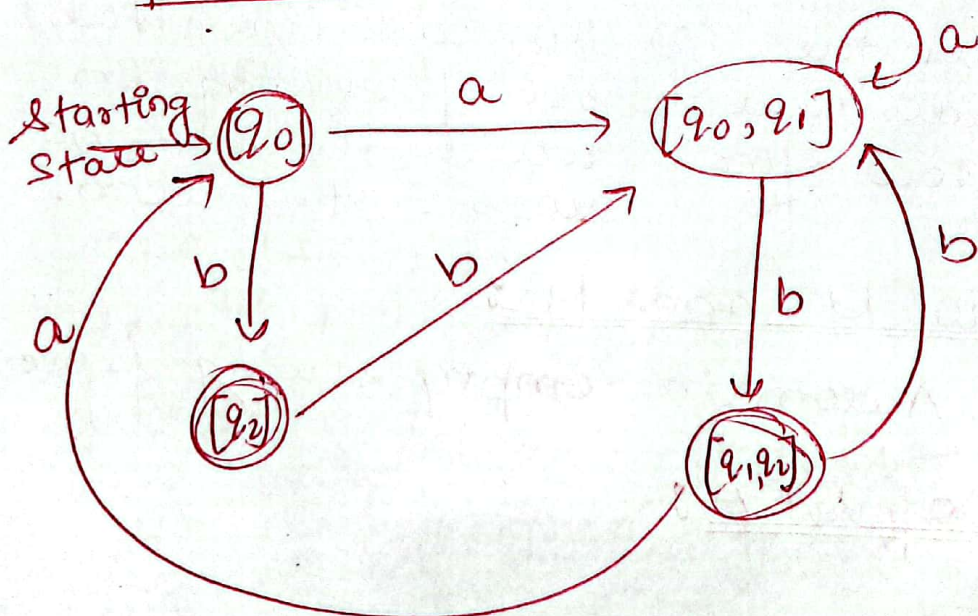
Find a DFA equivalent to NFA $M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$ where δ is defined as follows -

Present state (Ps)	a	b
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_2\}$
q_1	$\{q_0\}$	$\{q_1\}$
$*q_2$	—	$\{q_0, q_1\}$

Solution :-

Let $M_1 = (Q_1, \Sigma, \delta_1, [q_0], F_1)$ be the equivalent DFA, where $\Sigma = \{a, b\}$, $q_0 = [q_0]$ starting state δ_1 is defined as follows :-

Present state (Ps)	a	b
$\rightarrow [q_0]$	$[q_0, q_1]$	$[q_2]$
$[q_0, q_1]$	$[q_0, q_1]$	$[q_1, q_2]$
$*[q_2]$	—	$[q_0, q_1]$
$*[q_1, q_2]$	$[q_0]$	$[q_0, q_1]$



$$Q_1 = \{[q_0], [q_0, q_1], [q_2], [q_1, q_2]\}$$

$$F_1 = \{[q_2], [q_1, q_2]\}$$

(27)

● Difference b/w L^* and L^+ :

let $\Sigma = \{a\}$.

L^* is a Kleene closure over alphabet $\Sigma = \{a\}$

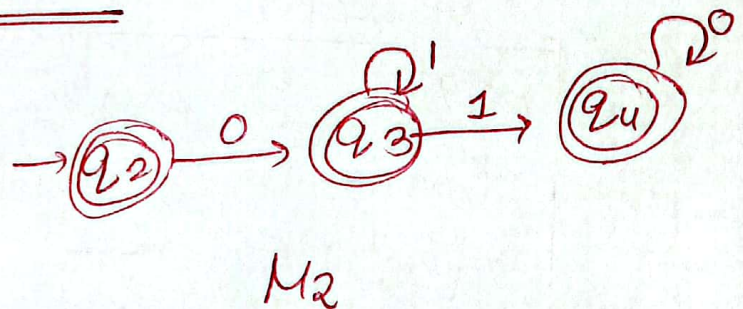
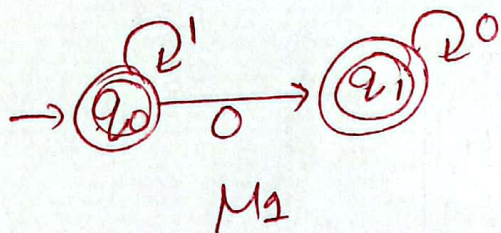
$L^* = \{\epsilon, a, aa, aaa, aaaa, \dots\}$.

L^+ is a positive closure over alphabet $\Sigma = \{a\}$.

$L^+ = \{a, aa, aaa, aaaa, \dots\}$.

Ques Show that the Automata M_1 and M_2 is given in figure are equivalent or not. Give the difference and similarity b/w the following FA.

● Difference b/w M_1 and M_2 :



(1) Have two states

(2) $L(M_1)$ will have the string in which there is no one is allow after 0 is encounter

(1) Have three states

(2) $L(M_2)$ will have the set of string in which there is no such restriction.

● Similarity b/w M_1 and M_2 :

In both the finite Automata empty string is accepted

Minimization of DFA :

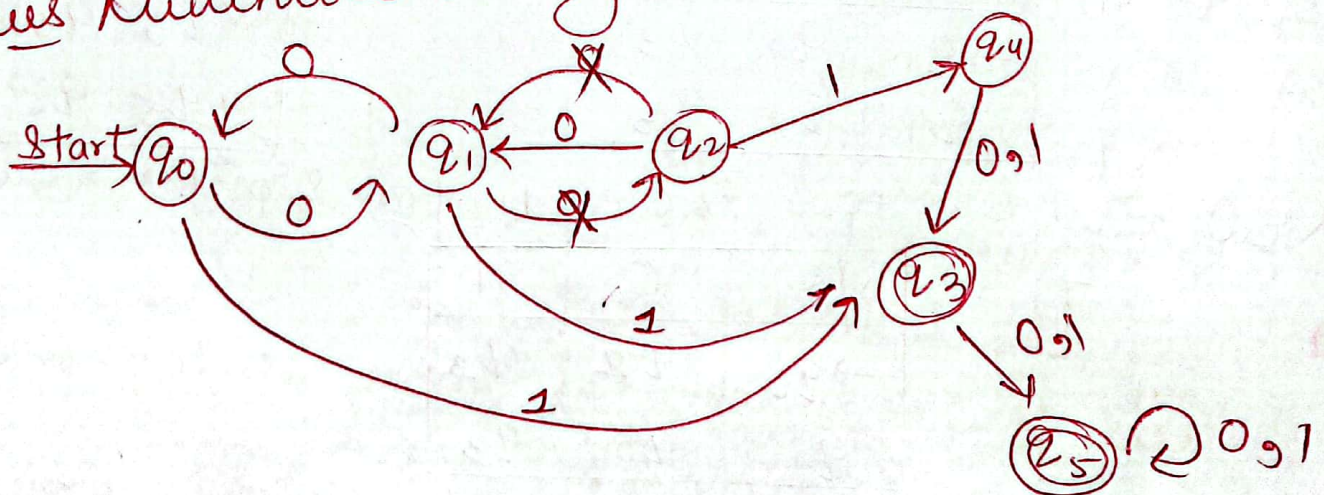
● Algorithm :

(1) Eliminate the dead state and unreachable states from the given DFA (if any)

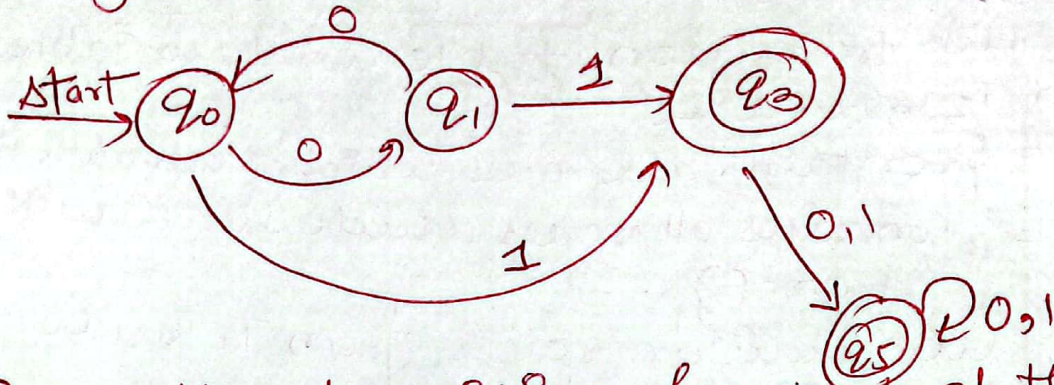
(2)

- ② Draw the transition table for next states after removing the dead state and unreachable states.
- ③ Split the transition table in two tables T_1 and T_2 ; where
 $T_1 = \{ \text{contains all rows which starts with states from } 0-F \}$
 $T_2 = \{ \text{contains all rows which starts with states from } F \}$
- ④ find the similar rows from T_1 such that:
 $\delta(q, a) = p$
 $\delta(r, a) = p$
- ⑤ Repeat step (IV) till there is no similar rows are available in T_1 .
- ⑥ Repeat the step (4) and step (5) for table T_2 also.
- ⑦ Now combined the reduced T_1 and T_2 .
 The combined transition table is transition table of minimized DFA.

Ques Minimize the given DFA



(1) Remove the unreachable state q_2 and q_4 from the given DFA.



(2) Draw the transition for rest of the states

δ/ϵ	0	1
$\rightarrow q_0$	q_1	q_3
q_1	q_0	q_3
$*q_3$	q_5	q_5
$*q_5$	q_5	q_5

(3) Split the transition table into two tables T_1 and T_2 .

$$T_1 =$$

δ/ϵ	0	1
$\rightarrow q_0$	q_1	q_3
q_1	q_0	q_3

$$T_2 =$$

δ/ϵ	0	1
$*q_3$	q_5	q_5
$*q_5$	q_5	q_5

(4) Now consider the table T_1 for similar rows.

$$T_1 =$$

δ/ϵ	0	1
$\rightarrow q_0$	q_1	q_3
q_1	q_0	q_3

(5) Now consider the table T_2 for finding similar rows.

$$T_2 =$$

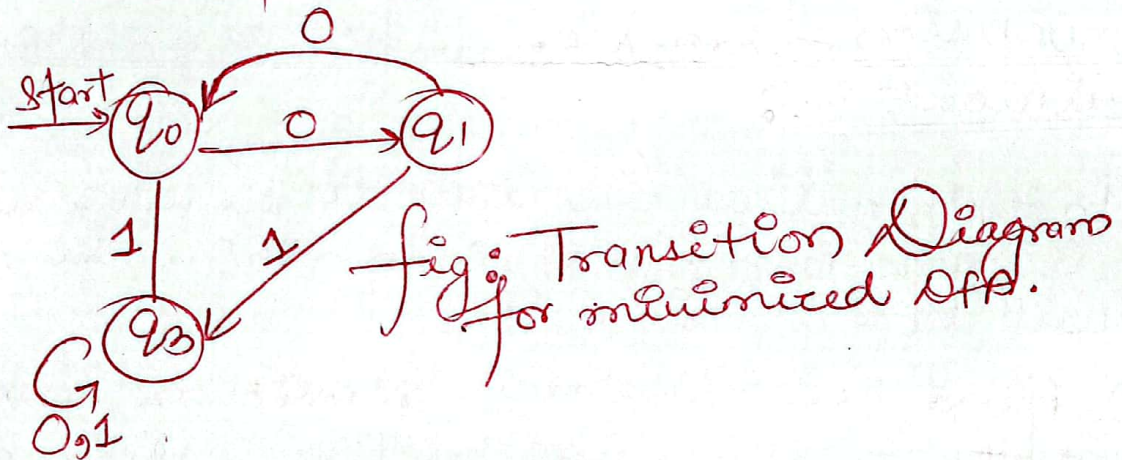
δ/ϵ	0	1
$*q_3$	q_5q_3	q_5q_3
$*q_5$	q_5	q_5

$$T_2 =$$

δ/ϵ	0	1
$*q_3$	q_3	q_3

⑥ Now combine the reduced table T_1 and T_2 the combine transition table is the transition table of minimized DFA.

δ/ϵ	0	1
$\rightarrow q_0$	q_1	q_3
q_1	q_0	q_3
$*q_3$	q_3	q_3

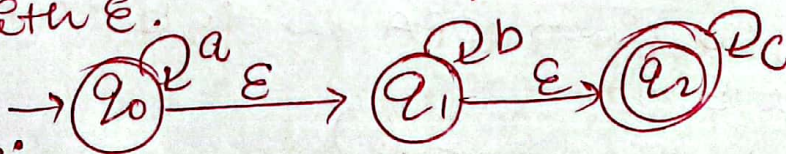


● Definition of ϵ -closure :

The ϵ -closure(p) is a set of all states which are reachable from state p on ϵ -transitions such that

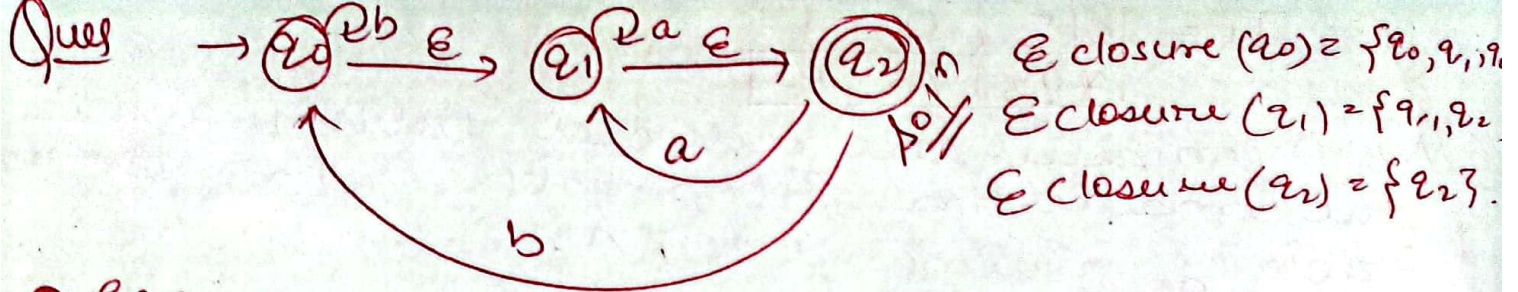
- (i) ϵ -closure(p) = p , where $p \in Q$
- (ii) if there exist ϵ closure(p) = $\{q\}$ & ϵ closure(q) = r then ϵ closure(p) = $\{q, r\}$.

Ques find ϵ -closure for the following NFA with ϵ .



Solution:

- ϵ -closure(q_0) = $\{q_0, q_1, q_2\}$ self state + ϵ reachable states
- ϵ -closure(q_1) = $\{q_1, q_2\}$
- ϵ -closure(q_2) = $\{q_2\}$



● Eliminating ϵ -transition :

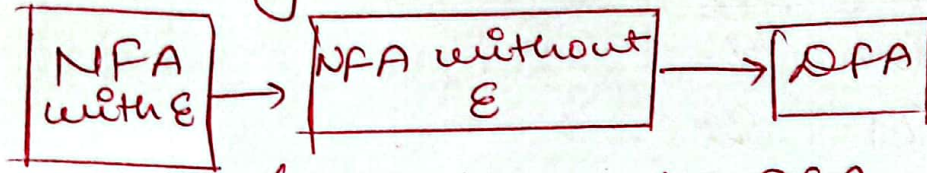


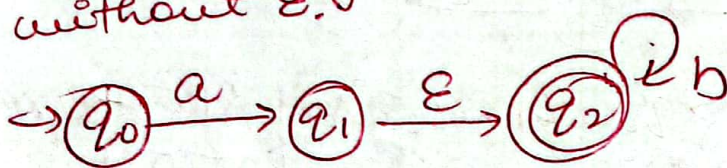
fig: Moves to DFA.

● Conversion from NFA with ϵ to NFA without ϵ :

In this method we try to remove all the ϵ transitions from given NFA. The method will be -

- (1) Find out all the ϵ transitions from each state from Q that will be called as ϵ -closure $\{q_i\}$, where $q_i \in Q$.
- (2) Then δ' transition can be obtained. The δ' transitions means an ϵ closure on δ moves.
- (3) Step 2 is repeated for each input symbol and for each state of given NFA.
- (4) Using the resultant state the transition table for equivalent NFA without ϵ can be built.

Ques Convert the given NFA with ϵ to NFA without ϵ .



26

Soln

$$\epsilon\text{-closure}(q_0) = \{q_0\}$$

$$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2\}.$$

Now the δ' transitions on each symbol is obtained

$$\begin{aligned}\delta'(q_0, a) &= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_0), a)) \\ &= \epsilon\text{-closure}(\delta(q_0, a)) \\ &= \epsilon\text{-closure}(q_1) \\ &= \{q_1, q_2\}\end{aligned}$$

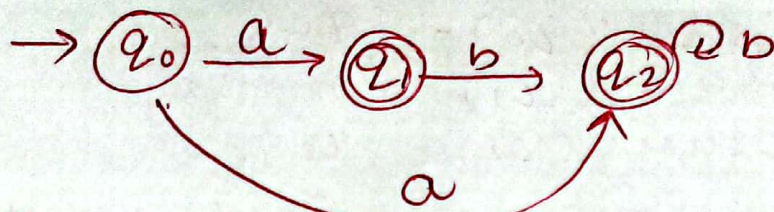
$$\begin{aligned}\delta'(q_0, b) &= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_0), b)) \\ &= \epsilon\text{-closure}(\delta(q_0, b)) \\ &= \epsilon\text{-closure}(\phi) \\ &= \phi.\end{aligned}$$

$$\begin{aligned}\delta'(q_1, a) &= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_1), a)) \\ &= \epsilon\text{-closure}(\delta(q_1, a) \cup \delta(q_2, a)) \\ &= \epsilon\text{-closure}(\phi) \\ &= \phi\end{aligned}$$

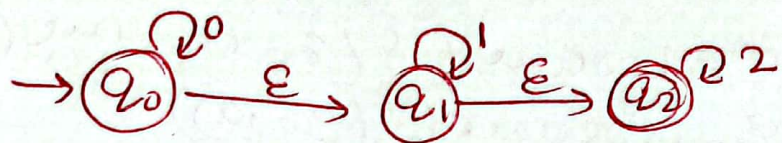
$$\begin{aligned}\delta'(q_1, b) &= \epsilon\text{-closure}(\delta(\epsilon\text{-closure}(q_1), b)) \\ &= \epsilon\text{-closure}(\delta(q_1, b) \cup \delta(q_2, b)) \\ &= \epsilon\text{-closure}(\phi \cup \delta(q_2, b)) \\ &= \epsilon\text{-closure}(\phi \cup q_2) \\ &= \epsilon\text{-closure}(q_2) \\ &= \{q_2\}.\end{aligned}$$

$\delta \mid \Sigma$	a	b
$\rightarrow q_0$	$\{q_1, q_2\}$	ϕ
$* q_1$	ϕ	$\{q_2\}$
$* q_2$	ϕ	$\{q_2\}$

Q2



Ques Convert the given NFA with ϵ to NFA without ϵ .



• Conversion of NFA with ϵ to DFA:

(1) Consider $M = (Q, \Sigma, \delta, q_0, F)$ is a NFA with ϵ . We have to convert this NFA with ϵ to equivalent DFA denoted by:

$M_0 = (Q_0, \Sigma, \delta_0, q_0, F_0)$ then obtain

ϵ -closure(q_0) = $\{p_1, p_2, p_3, \dots, p_n\}$ then $[p_1, p_2, p_3, \dots, p_n]$ becomes a start state of DFA

Now $[p_1, p_2, p_3, \dots, p_n] \in Q_0$

(2) We will obtain δ transition on $[p_1, p_2, p_3, \dots, p_n]$ for each input $\delta_0([p_1, p_2, p_3, \dots, p_n], a) = \epsilon$ -closure

$(\delta(p_1, a) \cup \delta(p_2, a) \cup \delta(p_3, a) \cup \delta(p_4, a) \dots \delta(p_n, a))$

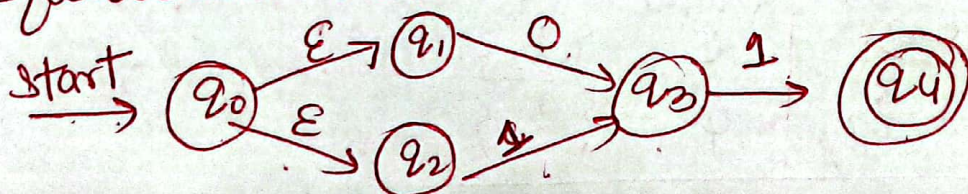
$\Rightarrow \bigcup_{i=1}^n \epsilon$ -closure(p_i, a)

where a is input $\in \Sigma$

(3) The states obtained $[p_1, p_2, p_3, \dots, p_n] \in Q_0$

The states containing final state in p_i is a final state in DFA.

Ques Convert the given NFA with ϵ to its equivalent DFA.



Solution: $E\text{-closure}(q_0) = \{q_0, q_1, q_2\}$
 $E\text{-closure}(q_1) = \{q_1\}$
 $E\text{-closure}(q_2) = \{q_2\}$
 $E\text{-closure}(q_3) = \{q_3\}$
 $E\text{-closure}(q_4) = \{q_4\}$

Now, let $E\text{-closure}(q_0) = \{q_0, q_1, q_2\}$ be state A

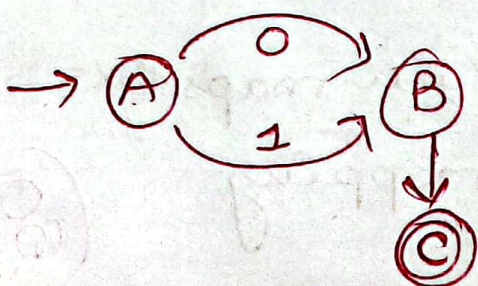
$$\begin{aligned}\delta'(A, 0) &= E\text{-closure}(\delta(q_0, q_1, q_2), 0) \\ &= E\text{-closure}(\delta(q_0, 0) \cup \delta(q_1, 0) \cup \delta(q_2, 0)) \\ &= E\text{-closure}(\emptyset \cup q_3 \cup \emptyset) \\ &= E\text{-closure}(q_3) \\ &= \{q_3\} \text{ be state B}\end{aligned}$$

$$\begin{aligned}\delta'(A, 1) &= E\text{-closure}(\delta(q_0, q_1, q_2), 1) \\ &= " (\delta(q_0, 1) \cup \delta(q_1, 1) \cup \delta(q_2, 1)) \\ &= " (\emptyset \cup \emptyset \cup q_3) \\ &= " (q_3) \\ &= \{q_3\} \text{ be state B}\end{aligned}$$

$$\begin{aligned}\delta'(B, 0) &= E\text{-closure}(\delta(q_3, 0)) \\ &= E\text{-closure}(\emptyset) \\ &= \emptyset\end{aligned}$$

$$\begin{aligned}\delta'(B, 1) &= E\text{-closure}(\delta(q_3, 1)) \\ &= E\text{-closure}(q_4) \\ &= \{q_4\} \text{ be state C}\end{aligned}$$

$$\begin{aligned}\delta'(C, 0) &= \emptyset \\ \delta'(C, 1) &= \emptyset\end{aligned}$$



δ/Σ	0	1
$\rightarrow A$	B	B
B	$\emptyset$	C
C	$\emptyset$	$\emptyset$

(29)

Finite state Machine (FSM) or Transducer:

A finite state m/c is similar to F.A except that it has the additional Capability of producing o/p.

$$FSM = FA + \text{Output Capability}$$

● Types of Finite state Machine:

① Moore M/c

② Mealy M/c

● Moore Machine:

If the output of FSM is dependent on present state only then this model of FSM is known as Moore m/c.

● Mealy Machine:

If the output of FSM is dependent on present state and present input then this model of FSM is known as Mealy m/c.

● Moore m/c definition:

A Moore m/c can be described by six tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$.

(1) Q is a finite set of states.

(2) Σ is the input alphabet.

(3) Δ is the output function alphabet

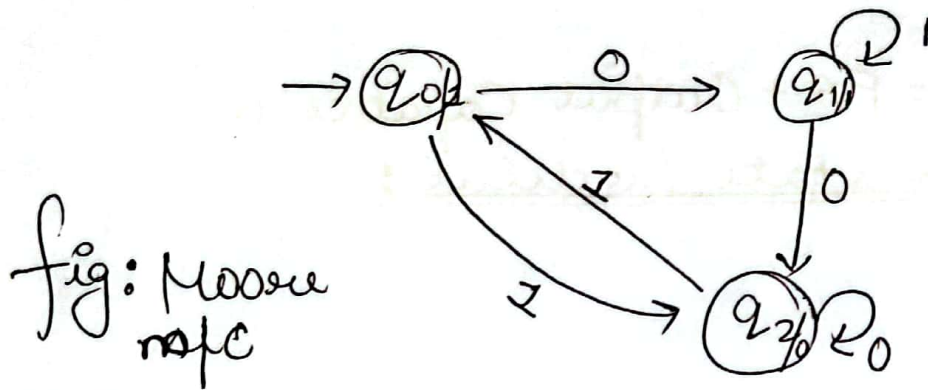
(4) q_0 is the initial state.

(5) δ is transition function which maps $Q \times \Sigma \rightarrow Q$

(6) λ is the output function mapping $Q \rightarrow \Delta$



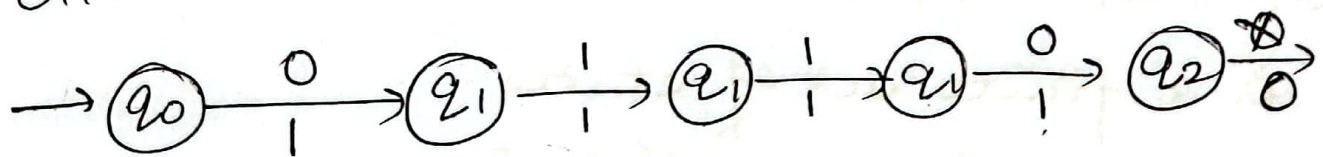
Example ~~Ques~~ Consider the Moore m/c shown in fig. below. Construct the transition table. What is the output for input string 0110?



Solⁿ Transition Table

Present state (PS)	Inputs		Outputs
	0	1	
	Next state (NS)	Next state (NS)	
→ q ₀	q ₁	q ₂	*
q ₁	q ₂	q ₀	1
q ₂	q ₂	q ₀	0

Now the transition sequence for the string 0110 is



So the o/p is 1110 for the input & the output is $\lambda(q_0) = 1$.

Note: - For Moore m/c if the input string is of length n the output string is of $n+1$

(31)

Ques What is the o/p for input string 0111?

Present
State

(22)

● Mealy Machine Definition:

A mealy m/c can be described by six tuple
 $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$

- (1) Q is finite and non-empty set of states
- (2) Σ is input alphabet.
- (3) Δ is output alphabet.
- (4) δ is transition funcⁿ which maps $Q \times \Sigma \rightarrow Q$
- (5) λ is output funcⁿ which maps $Q \times \Sigma \rightarrow \Delta$
- (6) q_0 is the initial state.

Ques Consider the Mealy m/c shown in fig. Construct the transition table & find o/p for input string 01010?

Solⁿ

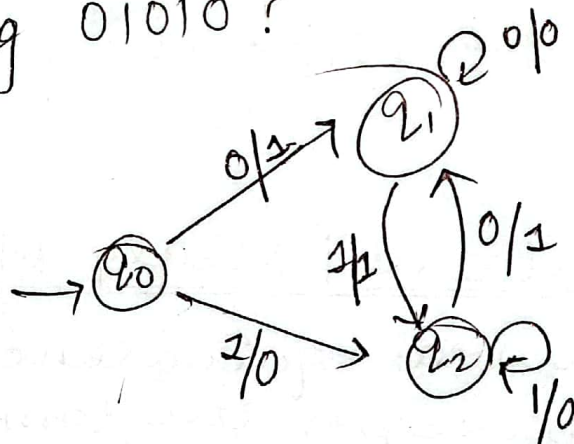
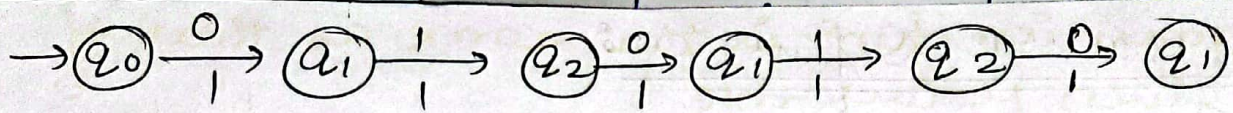


fig: Mealy Machine.

PS	Inputs			
	0		1	
	NS	O/P	NS	O/P
→ q ₀	q ₁	1	q ₂	0
q ₁	q ₁	0	q ₂	1
q ₂	q ₁	1	q ₂	0

Now the transition sequence for the input string 01010

33

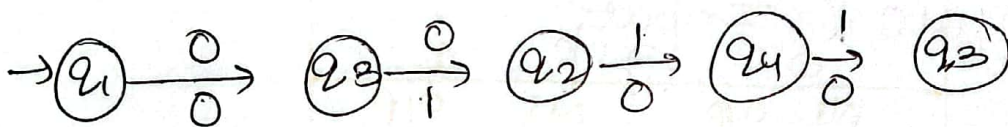


O/p $\rightarrow$ 11111

Ques Find the output for input string 0011?

Solⁿ

Ps	Inputs			
	0		1	
	Ns	O/P	Ns	O/P
$\rightarrow q_1$	q_3	0	q_2	0
q_2	q_1	1	q_4	0
q_3	q_2	1	q_1	1
q_4	q_4	1	q_3	0



O/p $\rightarrow$ 0100.

● Equivalence of Moore & Mealy MC:

The meaning of word equivalence is the two m/c's which accepts the same lang. Hence equivalence of Moore & Mealy m/c means both the m/c generate same O/P string or same I/P string. Procedure for transforming a Moore m/c to corresponding Mealy m/c:

● Construction:

- ① We have to define O/p funcⁿ λ' for Mealy m/c as a funcⁿ of present st^s. We define $\lambda'(q, a) = \lambda(\delta(q, a))$ for all states q and input symbol a .

(24)

2) The transition func is the same as that of the given Moore m/c

Ques Construct a Mealy m/c which is equivalent to the Moore m/c given in transition table.

Soln.

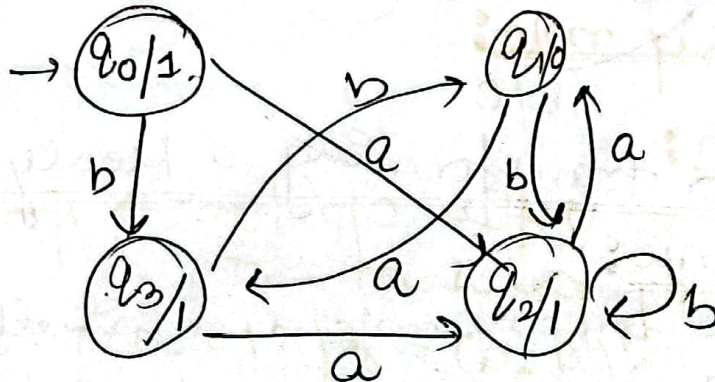
Present State (PS)	Inputs		O/P
	0 NS	1 NS	
→ q ₀	q ₃	q ₁	0
q ₁	q ₁	q ₂	1
q ₂	q ₂	q ₃	0
q ₃	q ₃	q ₀	0

Present state (PS)	Inputs			
	0		1	
	NS	O/P	NS	O/P.
→ q ₀	q ₃	0	q ₁	1
q ₁	q ₁	1	q ₂	0
q ₂	q ₂	0	q ₃	0
q ₃	q ₃	0	q ₀	0

① $\lambda'(q_0, 0) = \lambda(\delta(q_0, 0)) = \lambda(q_3) = 0$ ② $\lambda'(q_0, 1) = \lambda(\delta(q_0, 1)) = \lambda(q_1) = 1$

Ques Change the given Moore m/c into Mealy m/c.

Soln.



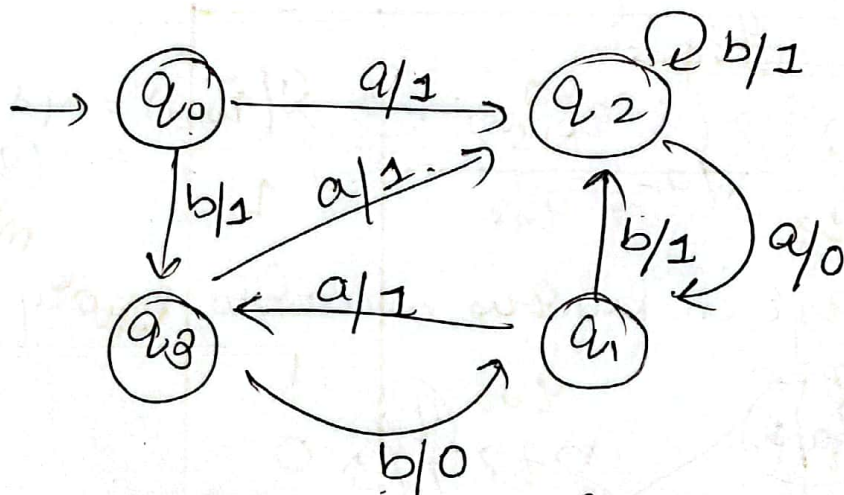
(36)

PS	Inputs		O/P
	a NS	b NS	
$\rightarrow q_0$	q_2	q_3	1
q_1	q_3	q_2	0
q_2	q_1	q_2	1
q_3	q_2	q_1	1

fig: Transition for Moore m/c

Present State (PS)	Input			
	a		b	
	NS	O/P	NS	O/P
$\rightarrow q_0$	q_2	1	q_3	1
q_1	q_3	1	q_2	1
q_2	q_1	0	q_2	1
q_3	q_2	1	q_1	0

fig: Transition Table for Mealy m/c



● Procedure for transforming a Mealy m/c to a Moore m/c:

Consider the mealy m/c described by the transition table. Construct the Moore m/c which is equivalent to Mealy m/c.

regular expression

Present state (PS)	Inputs			
	0		1	
	NS	O/P	NS	O/P
→ q ₁	q ₃	0	q ₂	0
q ₂	q ₁	1	q ₄	0
q ₃	q ₂	1	q ₁	1
q ₄	q ₄	1	q ₃	0

Mealy m/c

(37)

Present state (PS)	Inputs			
	0		1	
	NS	O/P	NS	O/P
→ q ₁	q ₃	0	q ₂₀	0
q ₂₀	q ₁	1	q ₄₀	0
q ₂₁	q ₁	1	q ₄₀	0
q ₃	q ₂₁	1	q ₁	1
q ₄₀	q ₄₁	1	q ₃	0
q ₄₁	q ₄₁	1	q ₃	0

q₂ < q₂₀
q₂₁

q₄ < q₄₀
q₄₁

Mealy m/c

Present state (PS)	Inputs		O/P
	0 NS	1 NS	
→ q ₁	q ₃	q ₂₀	1
q ₂₀	q ₁	q ₄₀	0
q ₂₁	q ₁	q ₄₀	1
q ₃	q ₂₁	q ₁	0
q ₄₀	q ₄₁	q ₃	0
q ₄₁	q ₄₁	q ₃	1

Moore m/c

Here we observe that the initial state q₁ is associated with output 1. This means with I/P 0.

we get an output 1. If the m/c start at state q_1 . To overcome this situation either we must neglect the response of a finite m/c to input ϵ or we must at a new ^{starting} state q_0 whose state transitions are identical with those of q_1 with whose output is 0.

Present State (PS)	Inputs		Output
	0 NS	1 NS	
$\rightarrow q_0$	q_3	q_{20}	0
q_1	q_3	q_{20}	1
q_{20}	q_1	q_{40}	0
q_{21}	q_1	q_{40}	1
q_3	q_{21}	q_1	0
q_{40}	q_{41}	q_3	0
q_{41}	q_{41}	q_3	1

Ques What are limitation & Application of FA?

Solⁿ Applications:

- (1) For the designing of lexical analysis of a Compiler.
- (2) Used in text editors.
- (3) For the implementations of spell checker.
- (4) For the designing of transducer.

Limitations:

- (1) FA can only count finite input.
- (2) It can have only string patterns.
- (3) Head movement is in only one direction.
- (4) Input tape is read only and only m/c it has is state to state.
- (5) There is no find and recognize set of binary string of 0's and 1's.
- (6) Set of strings over " $\{ \epsilon \text{ and } 0 \}$ " & to have balanced parenthesis.